

FOLHA DE EXERCÍCIOS: LISTAS E ESTRUTURAS MUTÁVEIS

Questão 1: Dada a lista inicial `numeros = [10, 20, 30, 40, 50, 60]`, escreva as linhas de código necessárias para:

- Alterar o primeiro elemento de forma direta para 15 e o último elemento para 99 (use indexação negativa).
- Substituir os elementos intermediários `[30, 40]` pelos novos valores `[100, 200]` de uma só vez utilizando fatiamento.
- O que acontece se executarmos o comando `print(numeros[6])`? Justifique com base no tipo de erro retornado pelo Python.

Questão 2: Explique, passo a passo, qual será o estado final da lista `valores = ['a', 'b', 'c', 'd', 'e', 'f']` após a execução sequencial de cada um dos seguintes comandos lógicos. Desenhe o estado da lista após cada exclusão:

- `del valores[1]`
- `del valores[2:4]`
- `valores.remove('f')`

Questão 3: Crie uma função chamada `analisar_notas_classe(lista_notas)` que receba uma lista contendo as notas de uma avaliação (ex: `[8.5, 6.0, 7.0, 9.5, 5.0]`). Sem utilizar a função nativa `append()`, a sua função deve:

- Criar um clone seguro da lista original de notas para evitar problemas de apelidamento na memória.
- Ordenar o clone em ordem crescente usando o método `.sort()`.
- Encontrar e exibir a menor nota e a maior nota a partir dos índices apropriados do clone.
- Calcular e retornar a média aritmética simples das notas contidas na lista.

Questão 4: Analise atentamente o código em Python apresentado a seguir:

- a) Qual será a saída impressa no terminal ao executarmos `print(lista_a)`? Explique o fenômeno computacional demonstrado (apelidamento/aliasing de listas na memória).
- b) Reescreva o código para garantir que a alteração do elemento em `lista_b` não afete de forma alguma o vetor original armazenado em `lista_a`. Demonstre como fazer isso usando fatiamento completo `[:]` ou o método `.copy()`.

Valor: 20 pontos.

```
lista_a = [1, 2, 3]
lista_b = lista_a
lista_b[0] = 99
print(lista_a)
```

Questão 5: As listas em Python são passadas para as funções através de referências na memória.

Considere a seguinte função proposta:

Se declararmos `meus_dados = [2, 4, 6]` no escopo principal do script e executarmos a chamada `duplicar_elementos(meus_dados)`, o que acontecerá com a variável `meus_dados` após a execução da função?

```
def duplicar_elementos(vetor):  
    for i in range(len(vetor)):  
        vetor[i] = vetor[i] * 2
```
